VCC | Assignment 3

Create a local VM and implement a mechanism to monitor resource usage. Configure it to auto-scale to a public cloud (e.g., GCP, AWS, or Azure) when resource usage exceeds 75%.

Submitted by:

Name: Manvendra Pratap Singh
Roll no. m25ai2122

Source code:
https://github.com/manvendrapratapsinghdev/IITJMaterial/tree/main/Sem-3/VCC/Assignments/3

Video Demo:

Project Document Flow

Project Creation

Prerequisites

- **VirtualBox** — Hypervisor for creating the local VM
- **Ubuntu 24.04.3 ISO** — https://mirrors.esto.network/ubuntu-releases/24.04.3/ubuntu-24.04.3-desktop-amd64.iso
- **AWS Account** — With IAM credentials for EC2 access
- **Source Code** — https://github.com/manvendrapratapsinghdev/IITJMaterial/tree/main/Sem-3/VCC/Assignments/3

Step-by-Step Implementation

Step 1: Install VirtualBox

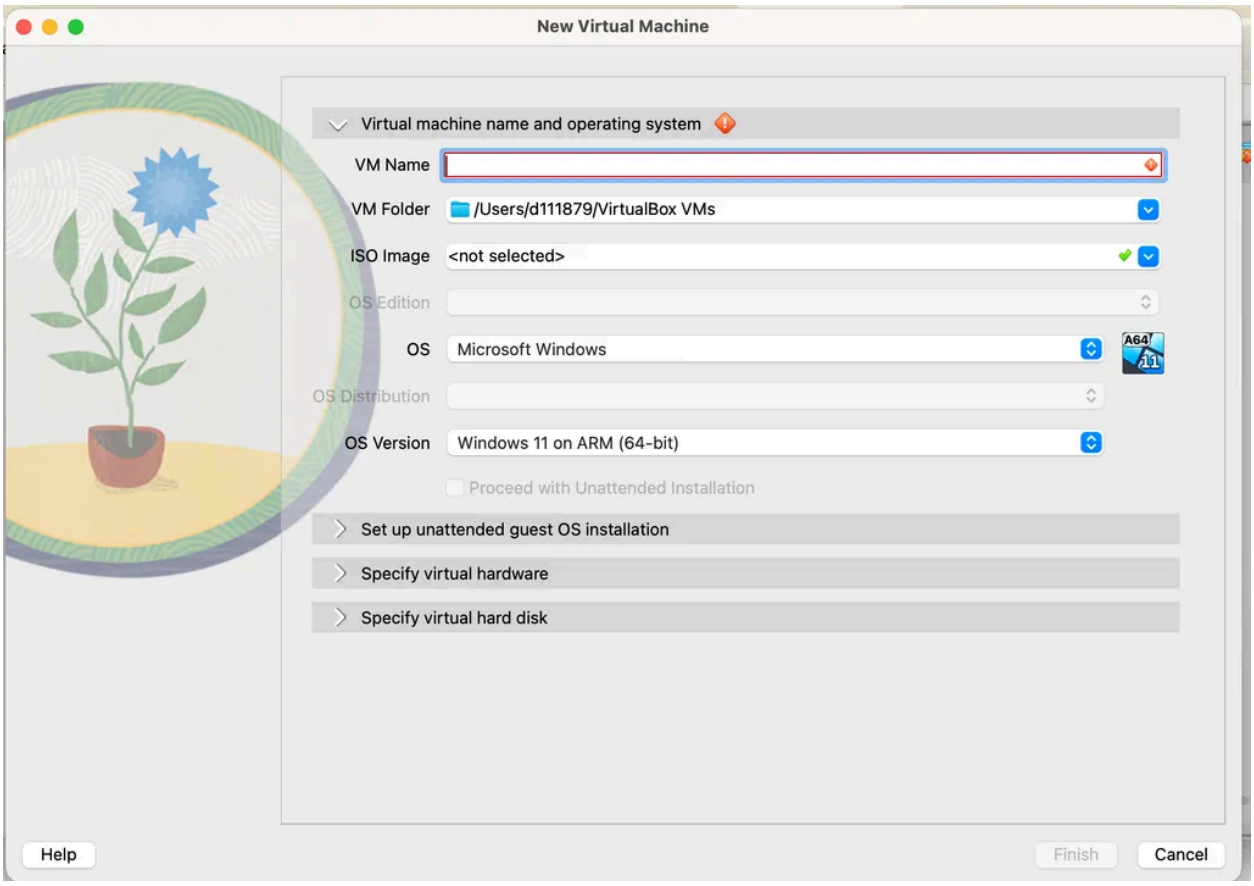
1. Open the **App Center** (or Software Center) on the host Ubuntu machine.
2. Search for **VirtualBox**.
3. Click **Install**.

Step 2: Create the Local VM (Assignment3-VM)

A. Create a New Virtual Machine

1. Open VirtualBox.
2. Press `Ctrl+N` to create a new machine.
3. Configure the VM with the following settings:

- **Name:** Assignment3-VM
- **Type:** Linux
- **Version:** Ubuntu (64-bit)
- **Memory (RAM):** 3072 MB
- **CPU Cores:** 1
- **Disk Type:** VDI (Dynamically Allocated)
- **Disk Size:** 15 GB



B. Configure Network Adapters

Each VM uses 2 Network Adapters:

- **Adapter 1 — NAT** → Purpose: Internet access (updates, packages)
- **Adapter 2 — Host-Only Adapter** (`vboxnet0`) → Purpose: Private communication between VMs

Steps:

1. Go to **VM** → **Settings** → **Network**.
2. Enable **Adapter 1** → Set to **NAT**.
3. Enable **Adapter 2** → Set to **Host-Only Adapter** (`vboxnet0`).

C. Install Ubuntu OS

1. Attach the Ubuntu 24.04.3 ISO to the VM.
2. Start the VM and follow the Ubuntu installation wizard.

D. Install Prerequisites on the VM

After OS installation, open a terminal and install the required tools:

```
sudo apt update
sudo apt install git curl unzip net-tools -y
```

Step 3: Install and Configure AWS CLI

```
# Download AWS CLI
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

# Unzip and install
unzip awscliv2.zip
sudo ./aws/install

# Verify installation
aws --version

# Configure credentials
aws configure
```

When prompted by `aws configure`, enter:

- **AWS Access Key ID** — from IAM console
- **AWS Secret Access Key** — from IAM console
- **Default region** — e.g., `ap-south-1`
- **Output format** — `json`

Step 4: AWS Configuration

1. **Create IAM Credentials:** Go to AWS IAM Console → Create an Access Key ID and Secret Access Key.
2. **Create a Security Group** with the following inbound rules:
   - **Port 22** (TCP) — SSH access
   - **Port 80** (TCP) — HTTP (Flask app)
   - **Port 443** (TCP) — HTTPS

   Note the **Security Group ID** (e.g., `sg-xxxxxxxx`) — it is required in `deploy_cloud.sh`.
3. **Identify the AMI ID** for the target region (e.g., `ami-05d2d839d4f73aafb` for Ubuntu).
4. **Create an SSH Key Pair** via the AWS Console for logging into the EC2 instance.

Step 5: Clone the Repository and Review Scripts

```
git clone https://github.com/manvendrapratapsinghdev/IITJMaterial.git
cd IITJMaterial/Sem-3/VCC/Assignments/3/auto-scale
```

The `auto-scale` directory contains 5 files:

- **monitor.py** — Monitors CPU and memory usage on the local VM using `psutil`. Triggers cloud deployment when usage exceeds 75%.
- **deploy_cloud.sh** — Launches an EC2 instance via AWS CLI, SSHs into it, installs Docker, builds the image, and runs the container.
- **app.py** — A Flask web application that exposes a GET endpoint on port 5000.
- **Dockerfile** — Defines the Docker image — uses Python 3.9, installs Flask, and runs `app.py`.
- **requirements.txt** — Python dependencies for the Flask app.

Step 6: Execute the Auto-Scaling Workflow

1. Navigate to the `auto-scale` directory inside the local VM:

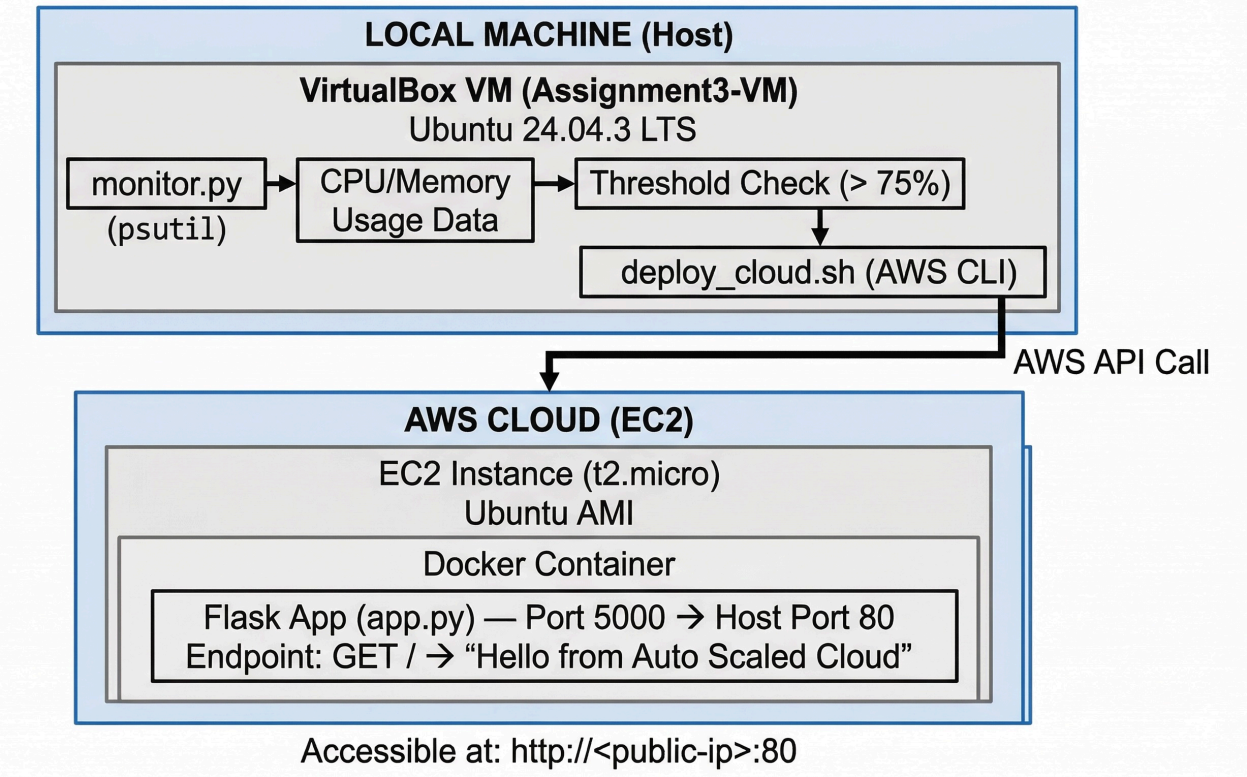
```
cd vcc/auto-scale
```

1. Run the monitoring script:

```
python3 monitor.py
```

1. The script continuously prints CPU and memory usage.
2. When either metric exceeds **75%**, the script automatically:
   - Prints `"Threshold exceeded. Scaling to cloud..."`
   - Executes `deploy_cloud.sh`
   - Launches an EC2 instance on AWS
   - Deploys the Flask app inside a Docker container
3. Once deployed, the application is accessible at EC2-Public-IP

Architecture Design



Flow

1. `monitor.py` continuously monitors CPU and memory usage on the local VM using `psutil`.
2. When either metric exceeds the 75% threshold, it triggers `deploy_cloud.sh`.
3. `deploy_cloud.sh` uses the AWS CLI to launch a new EC2 instance.
4. Once the instance is running, it SSHs into the instance, installs Docker, clones the repo, builds the Docker image, and runs the Flask application container.
5. The application becomes accessible on the EC2 instance's public IP on port 80.

Microservices (Additional Components)

vm-gateway (Port 3000)

An Express.js API gateway that forwards requests to backend services:

- `GET /api/service1` → forwards to `vm-service-1`
- `GET /api/service2` → forwards to `vm-service-2`

Service hosts and ports are configurable via environment variables (`SERVICE1_HOST`, `SERVICE1_PORT`, `SERVICE2_HOST`, `SERVICE2_PORT`).

---

vm-service-1 (Port 3001)

A minimal Express.js microservice exposing `GET /service1` that returns the service name, hostname, host IP, and timestamp.

---

vm-service-2 (Port 3002)

A minimal Express.js microservice exposing `GET /service2` with the same response structure as service-1.

Source Code Repository

- Repository: https://github.com/manvendrapratapsinghdev/IITJMaterial
- Auto-scale scripts: https://github.com/manvendrapratapsinghdev/IITJMaterial/tree/main/Sem-3/VCC/Assignments/3/auto-scale

---

Plagiarism Declaration

I hereby declare that this implementation and documentation titled **"Local VM Resource Monitoring with Automatic Cloud Scaling"** is my original work. No part of this submission has been copied from any external source. All scripts, configurations, and documentation were developed and implemented independently for academic purposes. The following open-source tools, libraries, and platforms were used in the implementation:

- **Python libraries:** Flask (web framework), psutil (system monitoring)
- **Node.js libraries:** Express.js (API gateway and microservices)
- **Infrastructure:** Docker (containerization), AWS CLI (cloud provisioning), VirtualBox (virtualization)

- **OS:** https://mirrors.esto.network/ubuntu-releases/24.04.3/ubuntu-24.04.3-desktop-amd64.iso
- **Cloud:** AWS EC2 (auto-scaling target)

Manvendra Pratap Singh (M25AI2122)